



VIT[®]
BHOPAL

Project Report

BookMyShow Movie Booking System

Name: Yash Sharma

Registration Number: 25BAI10305

Course: (CSE-2006) Programming in Java

Table of Contents

Sr. No.	Section Title	Page No.
1	Introduction	4
2	Problem Statement	5
3	Objectives	6
4	Functional Requirements	8
5	Non-Functional Requirements	10
6	System Architecture	11
7	Design Diagrams	13
8	Design Decisions & Rationale	24
9	Implementation Details	25
10	Screenshots / Results	26
11	Testing Approach	32
12	Challenges Faced	33
13	Learnings & Key Takeaways	34
14	Future Enhancements	35

15	References	36
----	------------	----

1. Introduction

The Multiplex Movie Booking System is a console-based Java program that mimics the real experience of booking movie tickets at a modern multiplex cinema. The Multiplex Movie Booking System was created for coursework to show skill in Java ideas such as object-oriented programming, collections framework, enumerations, lambda expressions and the Stream API.

The Multiplex Movie Booking System has two roles – a regular User and an Admin – each with its own portal and features. The User can look at a list of movies now playing select a showtime pick seats from a tiered layout (VIP, Premium, Executive, Normal) optionally add snacks from the in-app snack bar use discount codes and finish booking with a tax-inclusive invoice. The Admin can handle the movie catalog set up shows on different screens, delete movies and see detailed revenue analytics for the whole multiplex.

The entire Multiplex Movie Booking System lives in one Java file (MultiplexBookingSystem.java). Uses in-memory data structures like Lists, Maps and Sets instead of an external database. The Multiplex Movie Booking System keeps the focus on software design, clean architecture and a smooth user experience, on the command line.

2. Problem Statement

In areas, especially in semi-urban and smaller cities multiplex cinemas still depend on manual ticket booking at counters. This outdated method causes problems. Long lines form at the ticket counter during busy hours and weekends. Customers often arrive only to find that seats are already sold out. There is no way for them to check seat availability before going to the cinema. That means many people end up making trips that're not worth it.

Another issue is pricing. Since everything is done by hand errors happen. Sometimes discounts are. Applied incorrectly especially when promotions are running. This creates confusion and frustration for customers who expect deals. Cinema managers also struggle without a system. They cannot easily track how money they are making how full their screens are or which movies are performing well. Without this data decisions about scheduling can be based on guesswork of facts. Adding showtimes for popular movies and removing older ones becomes a challenge. When a new movie takes off extra shows are needed quickly.. With manual tracking shifting schedules is slow and messy.

Even ordering snacks is an afterthought. After buying tickets customers have to wait in another line just to get food. There is no way to order snacks along with tickets. This project aims to fix all these issues. It builds a self-service digital booking platform. Users can browse movies pick seats and book tickets on their own. The interface shows real-time seat availability, clear prices and full transparency. At the time the system gives cinema administrators powerful tools. They can manage what movies are showing adjust showtimes and see real-time revenue and occupancy numbers. With data they can make smarter choices about which movies to keep and how often to run them. The goal is simple: make movie-going easier for customers and smoother, for theater owners.

3. Objectives

In areas, especially in semi-urban places and smaller cities big movie theaters still use old-fashioned ticket counters. This causes a lot of problems.

- People have to wait in lines at the ticket desk especially when things are busy and on weekends.
- Customers can't see which seats are available before they get to the theater so they might come all the way. Find no seats.
- Mistakes happen with prices and discounts when special deals are going on.
- The people who run the theaters don't have a place to check how much money is being made how full the seats are and how well each show is doing.
- It's hard to change the schedule when a popular movie needs showtimes and an older one needs to be taken off.
- There's no way to order snacks while booking tickets. People have to go to another line for food after they buy their tickets.

This project solves these problems by creating a booking system where people can easily look at movies pick seats and book tickets on their own. Everything is clear from the price to the seats. What's available. All this happens on one screen. On the end the system gives tools to manage movies and shows plus real-time reports on money made. This helps managers make decisions about which movies to show and when.

1. Create a console application using Java and good object-oriented programming to show clean software design.
2. Make two parts of the system (User and Admin) with separate roles, menus and abilities to match how things work in real life.
3. Build a seat system with levels (VIP, Premium, Executive, Normal) and prices that change depending on the seat.
4. Make a seat map that shows which seats are taken and which are free so people can choose the one.
5. Add a system for using discount codes that can be used only once for special offers.
6. Include a part that adds taxes and extra fees (18% GST and a convenience fee) to make the billing look real.

7. Give the admin a dashboard that shows money made how full the theater is and how well each movie is doing.



8. Fill the app with a looking list of 8 movies and more, than 25 showtimes across 8 screens.

9. Add a snack section so people can buy food while they are booking their tickets.

10. Make sure the whole ticket process works, from booking to seeing your tickets and even canceling with a refund.

4. Functional Requirements



The system shall support the following functional capabilities:

4.1 User Portal

ID	Requirement	Description
FR-U01	Browse Movies	The system shall display all currently showing movies with their ID, title, and genre.
FR-U02	View Showtimes	Upon selecting a movie, the system shall list all scheduled shows sorted by time, showing screen name and time slot.
FR-U03	Seat Selection	The system shall display a visual seat map with tier-based pricing and real-time availability. Users can select one or more seats by entering seat IDs.
FR-U04	Snack Ordering	During booking, the system shall present a snack bar menu. Users can add multiple items before proceeding to checkout.
FR-U05	Promo Code Application	The system shall accept promotional codes. Permanent codes (WELCOME10, FESTIVAL20) provide percentage discounts. Single-use bulk reward codes provide 20% off.
FR-U06	Invoice Generation	The system shall calculate and display a detailed invoice with seat subtotal, snack subtotal, discount, GST (18%), convenience fee (Rs. 35), and grand total.
FR-U07	Ticket Wallet	The system shall store all active tickets in a user wallet. Users can view all their booked tickets at any time.
FR-U08	Ticket Cancellation	The system shall allow cancellation by ticket ID.

		Cancelled tickets free the seats, refund the amount, and revoke any associated promo codes.
--	--	---

4.2 Admin Portal

ID	Requirement	Description
FR-A01	Revenue Analytics	The system shall display per-movie, per-show revenue with seats sold count and a grand platform total.
FR-A02	Add Movie	The admin shall be able to add new movies by specifying title and genre. A unique ID is auto-generated.
FR-A03	Add Show	The admin shall be able to schedule a new show for an existing movie by specifying screen name and time.
FR-A04	Remove Movie	The admin shall be able to remove a movie, which cascades to delete all associated shows and booked tickets.

5. Non-Functional Requirements

NFR-01: Performance

The application shall respond to all user inputs within 1 second. This timing excludes the 1.5-second payment delay. All data operations use in-memory collections to ensure $O(n)$ or lookup times.

NFR-02: Usability

The CLI shall use menus, formatted tables, visual seat maps created with ASCII art and emoji indicators to provide a polished and intuitive user experience. Error messages must be clear and descriptive to guide users

NFR-03: Reliability

The application shall handle all inputs gracefully. This includes menu choices, invalid seat IDs and duplicate bookings. The system must not crash under any of these conditions. Input validation is applied at every interaction point to ensure robustness.

NFR-04: Maintainability

The codebase follows a separation of concerns. Entity classes such as Movie, Show, Seat and Ticket are kept separate from enumerations like SeatTier and SnackItem. The main controller manages the flow. This modular structure makes it easy to extend and maintain.

NFR-05: Scalability

Although the current version uses in-memory data the entity classes are designed to be adapted to a database-backed persistence layer. Minimal refactoring is needed to transition to a storage system.

NFR-06: Portability

The application runs on any platform with JDK 17 or higher. It has zero dependencies. Only the Java standard library is used, ensuring compatibility and easy deployment.

NFR-07: Data Integrity

Seat booking and unbooking operations are tightly coupled. Cancelling a ticket must properly release the seat deduct revenue remove the ticket from the wallet and revoke any generated promo codes. All actions are synchronized to maintain data consistency.

6. System Architecture

The Multiplex Movie Booking System uses a layered architecture pattern. The code is logically organized into three layers:

6.1 Presentation Layer

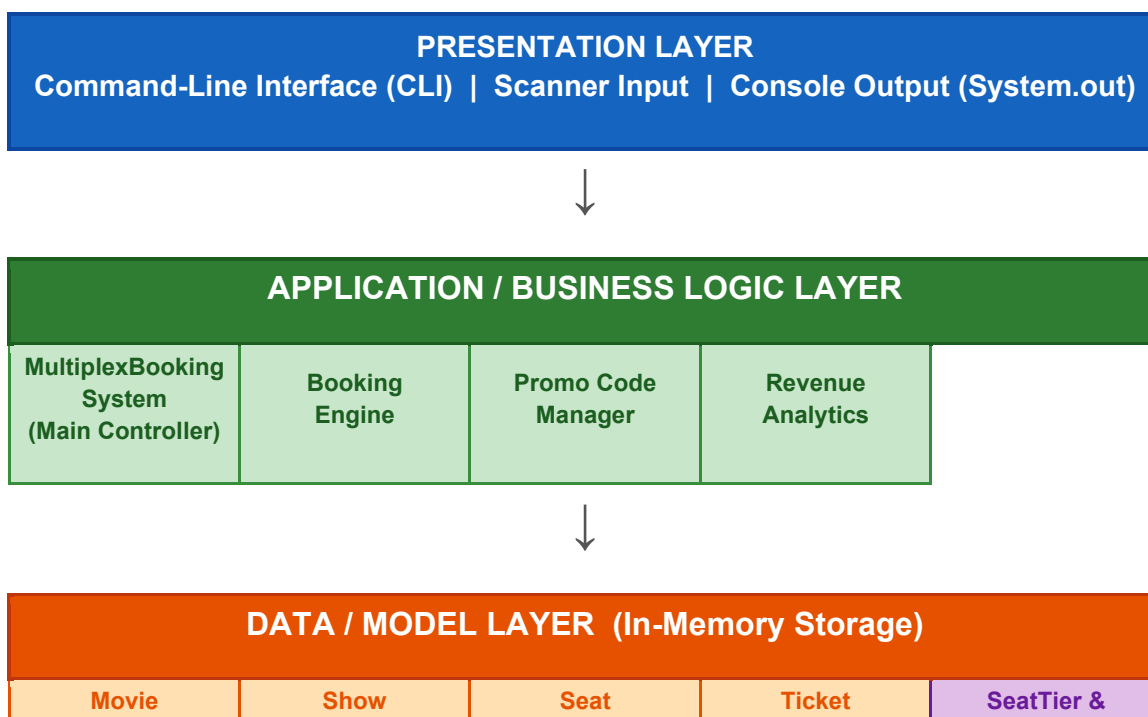
The Presentation Layer is a command-line interface that the end user interacts with. It handles all input and output operations. The Multiplex Movie Booking System reads user input with `Java's Scanner` class and displays results with `System.out`. The Multiplex Movie Booking System shows the seat map, ticket receipts and analytics dashboards as formatted console output.

6.2 Application / Business Logic Layer

The Multiplex Movie Booking System class acts as the controller. It runs the flow from portal selection to booking, cancellation and admin operations. The Multiplex Movie Booking System follows business rules such as seat validation promo code lifecycle management, tax calculation with 18% GST plus a Rs. 35 Fee, bulk booking reward generation and cascading deletion.

6.3 Data / Model Layer

Entity classes `Movie`, `Show`, `Seat` and `Ticket` along with enumerations `SeatTier` and `SnackItem` form the data layer. The Multiplex Movie Booking System stores all data, in memory using `ArrayList`, `HashMap` and `HashSet` collections. Each `Show` automatically generates 36 tiered seats during construction.





System Architecture Diagram (Layered Architecture)

7. Design Diagrams

7.1 Use Case Diagram

The use case diagram shows two actors – User and Admin – and how they interact. The User works with eight use cases that cover the booking lifecycle. The Admin works with four use cases that focus on content management and analytics.

System Boundary: Multiplex Movie Booking System

ACTOR: USER	USER USE CASES
→	UC-01: Browse Movies
→	UC-02: View Show Schedules
→	UC-03: Select Seats from Seat Map
→	UC-04: Order Snacks from Snack Bar
→	UC-05: Apply Promotional Code
→	UC-06: Book Ticket & Process Payment
→	UC-07: View My Tickets (Wallet)
→	UC-08: Cancel Ticket & Get Refund

ACTOR: ADMIN	ADMIN USE CASES
→	UC-09: View Multiplex Revenue Analytics
→	UC-10: Add New Movie to Catalogue
→	UC-11: Schedule New Show for Movie



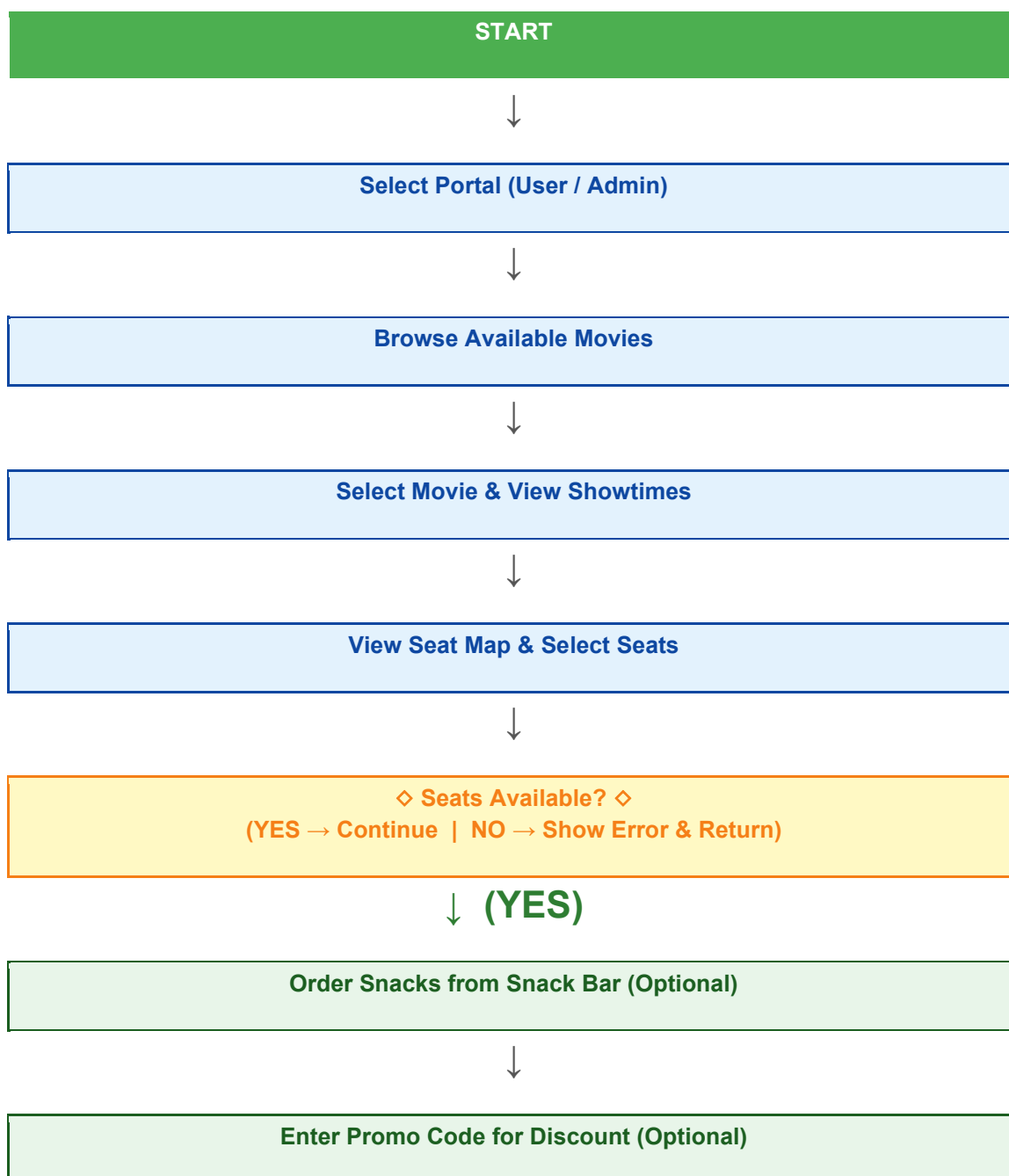
UC-12: Remove Movie (Cascade Delete)

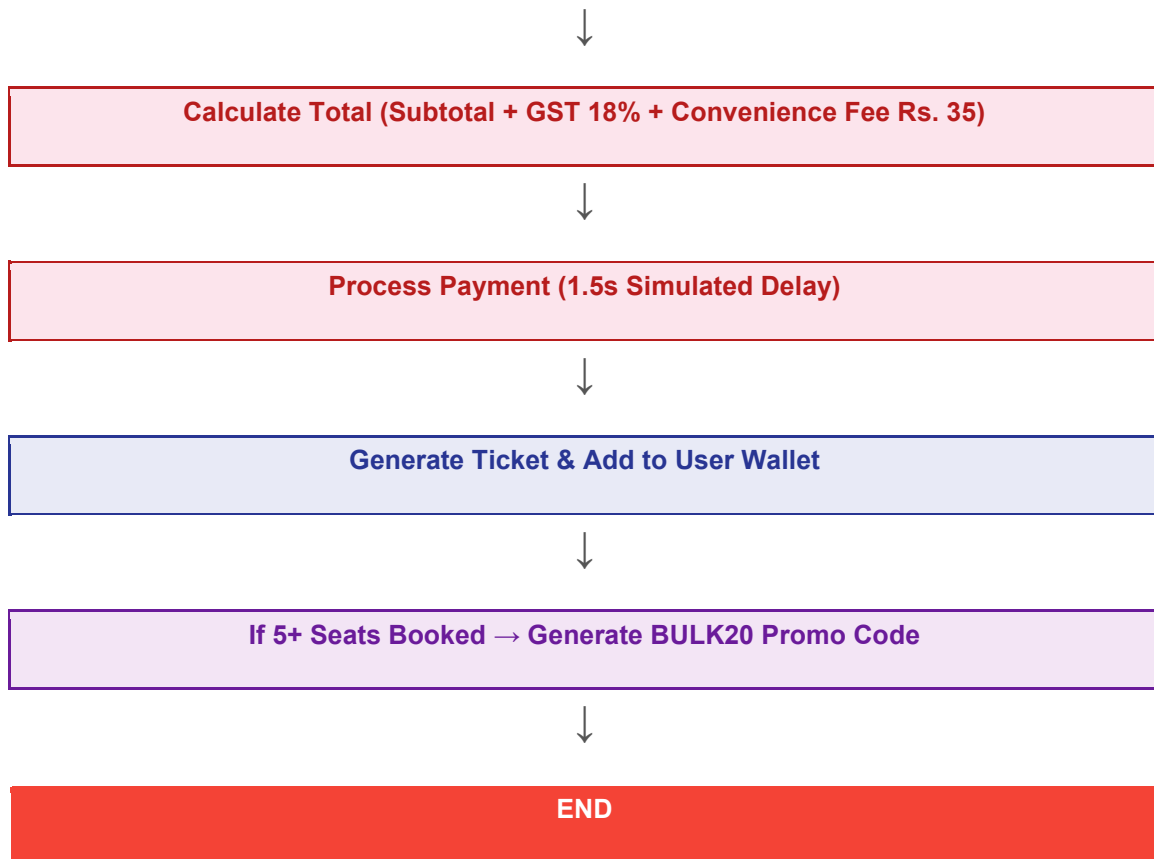
14

Use Case Diagram

7.2 Workflow / Process Flow Diagram

The workflow diagram follows the entire ticket booking journey from beginning to end. The User starts by choosing the User portal looks through movies selects a showtime checks the seat map and chooses seats. If the chosen seats are not available the system displays an error. If the seats are available the User can add snacks. Use a promo code if they wish. The system then adds GST to the total processes the payment creates a ticket and stores it in the wallet.

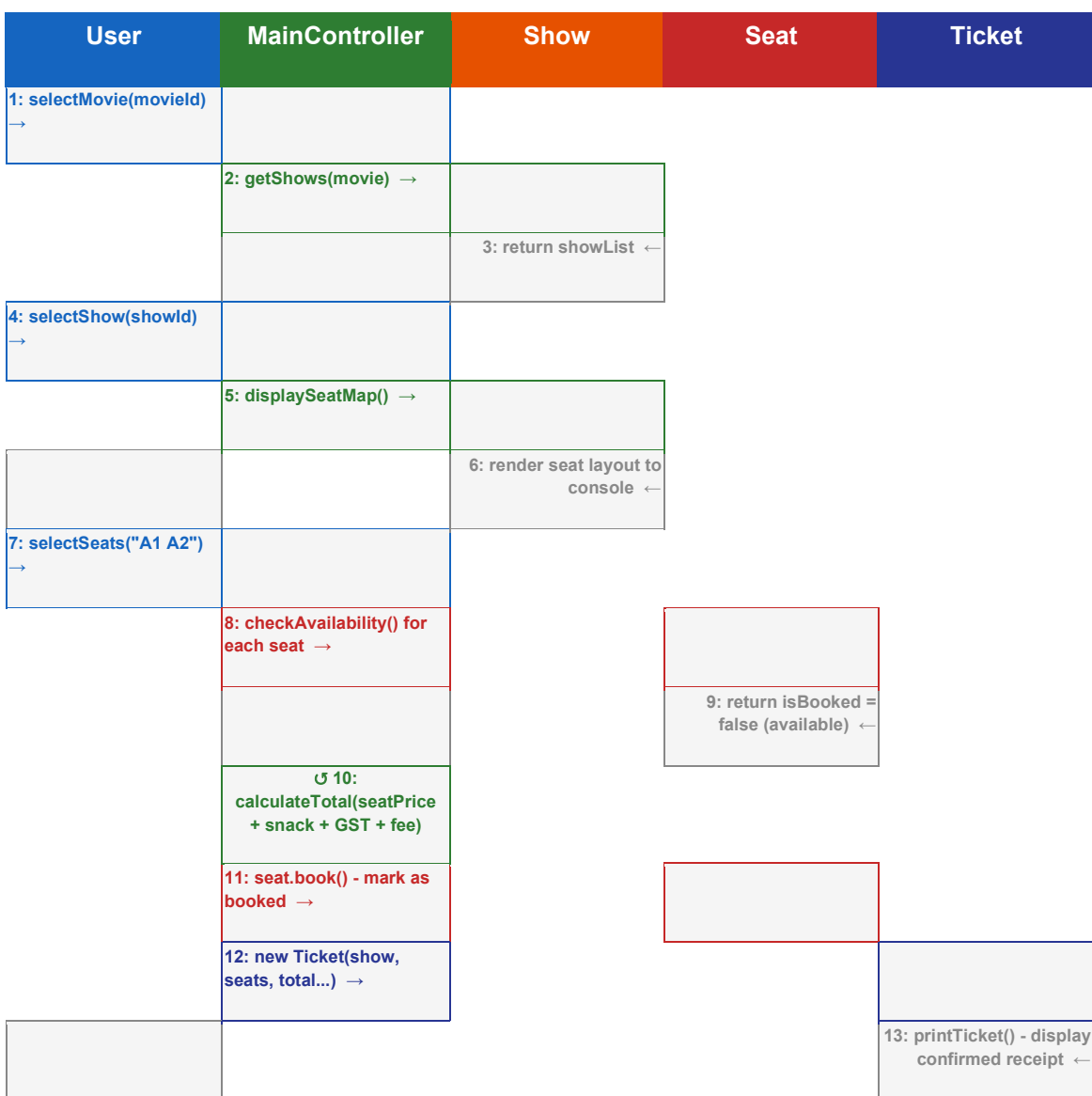




Booking Workflow / Process Flow Diagram

7.3 Sequence Diagram

The sequence diagram displays how objects talk during a booking transaction. The User starts by picking a movie. The controller then pulls show information shows the seat map checks that seats are available calculates the bill reserves the seats makes a ticket and prints the confirmation..



Sequence Diagram - Ticket Booking Flow

7.4 Class / Component Diagram

The class diagram lists every entity in the system. How they are linked. Each class box contains the class name, its attributes with types and its methods. Links between classes are written below the diagram, with cardinality notes.

MultiplexBookingSystem (Main Controller)

- movies : List<Movie>
- shows : List<Show>
- userWallet : List<Ticket>
- permanentPromoCodes : Map<String, Double>
- singleUsePromoCodes : Map<String, Double>
- usedPromoCodes : Set<String>
- scanner : Scanner

- + main(String[] args) : void
- + runPortalSelection() : void
- + runUserMenu() : void
- + runAdminMenu() : void
- + processBookingFlow() : void
- + processSnackBar() : List<SnackBarItem>
- + processCancellation() : void
- + displayRevenueAnalytics() : void
- + seedInitialData() : void

Movie

- movieId : String
- title : String
- genre : String

- + getMovieId() : String
- + getTitle() : String
- + getGenre() : String
- + equals(Object) : boolean
- + hashCode() : int

Show

- showId : String
- movie : Movie
- screenName : String
- time : String
- seats : List<Seat>

- totalRevenueGenerated : double

+ getShowId() : String
 + getMovie() : Movie
 + getSeats() : List<Seat>
 + displaySeatMap() : void
 + addRevenue(double) : void
 + deductRevenue(double) : void
 - generateTieredSeats() : List<Seat>

Seat

- seatId : String
 - tier : SeatTier
 - isBooked : boolean

+ getSeatId() : String
 + getTier() : SeatTier
 + isBooked() : boolean
 + book() : void
 + unbook() : void

Ticket

- ticketId : String (UUID)
 - show : Show
 - bookedSeats : List<Seat>
 - snacks : List<SnackItem>
 - seatTotal : double
 - snackTotal : double
 - discountApplied : double
 - gstAmount : double
 - convenienceFee : double
 - grandTotal : double
 - earnedPromoCode : String

+ getTicketId() : String
 + getGrandTotal() : double
 + getBookedSeats() : List<Seat>
 + getEarnedPromoCode() : String
 + printTicket() : void

«enumeration» SeatTier

VIP (Rs. 500)
 PREMIUM (Rs. 350)
 EXECUTIVE (Rs. 250)
 NORMAL (Rs. 150)

+ getPrice() : double

«enumeration» SnackItem

POPCORN - Large Caramel Popcorn (Rs. 220)
 NACHOS - Jalapeno Cheese Nachos (Rs. 180)
 COKE - Fountain Coke (Rs. 120)
 WATER - Mineral Water (Rs. 60)

+ getDescription() : String
 + getPrice() : double

Class Relationships

MultiplexBookingSystem —has many—► Movie (1 : *)

MultiplexBookingSystem —has many—► Show (1 : *)

MultiplexBookingSystem —has many—► Ticket (userWallet) (1 : *)

Show —references—► Movie (many : 1)

Show —contains—► Seat (1 : 36 composition)

Ticket —references—► Show (many : 1)

Ticket —contains—► Seat (many : many)

Ticket —contains—► SnackItem (many : many)

Seat —uses—► SeatTier (many : 1 enum reference)

Class Diagram with Relationships

7.5 ER Diagram (Data Model)

Although this application uses storage that stays in memory of a regular database, the way the data is organized follows a clear way of showing how different parts are connected. Each part of the data has names for the fields what kind of information they hold and rules that apply (PK means Primary Key FK means Foreign Key). The table that shows how the parts are connected explains how connections there are and what kind of connections they are.

MOVIE		
movieId	String	PK
title	String	
genre	String	

SHOW		
showId	String	PK
movieId	String	FK → MOVIE
screenName	String	
time	String	
totalRevenueGenerated	double	

SEAT		
seatId	String	PK
showId	String	FK → SHOW
tier	SeatTier	Enum ref
isBooked	boolean	Default: false

TICKET		
--------	--	--

ticketId	String (UUID)	PK
showId	String	FK → SHOW
bookedSeats	List<Seat>	FK → SEAT (*)
snacks	List<SnackItem>	
seatTotal	double	
snackTotal	double	
discountApplied	double	
gstAmount	double	
convenienceFee	double	
grandTotal	double	
earnedPromoCode	String	Nullable

SNACK_ITEM (Enum)

name	String	Enum constant
description	String	
price	double	

SEAT_TIER (Enum)

name	String	VIP/PREMIUM/EXECUTIVE/NORMAL
price	double	500/350/250/150

Entity Relationships

Entity A	Cardinality	Entity B	Cardinality	Description
MOVIE	1	SHOW	*	One Movie can have many Shows scheduled across different screens and times.
SHOW	1	SEAT	36	One Show contains

				exactly 36 Seats (6 VIP + 12 Premium + 12 Executive + 6 Normal). Seats are generated when a Show is created (composition).
TICKET	*	SHOW	1	Many Tickets can reference one Show. Each Ticket is for a specific show screening.
TICKET	*	SEAT	*	A Ticket references one or more Seats. A Seat can only belong to one Ticket at a time.
TICKET	*	SNACK_ITEM	*	A Ticket can optionally include multiple Snack Items ordered during booking.
SEAT	*	SEAT_TIER	1	Each Seat has exactly one SeatTier that determines its price.

Entity-Relationship (ER) Diagram with Relationship Table

8. Design Decisions & Rationale

Single-File Architecture

All of the application is in just one Java file. Even though it is more usual to have files this way makes it simple to compile and run without needing special tools. Even though it is all in one file the code is organized clearly into enums, classes for the data and the part that handles the actions.

In-Memory Storage of a Database

Using lists, maps and sets instead of a database keeps the project all by itself with no outside help. The way the data is set up is made so that changing to JPA/Hibernate would take few changes.

Enums for SeatTier and SnackItem

Java enums were used of simple words or fixed values. Enums make sure the right type is used have built-in ways to go through all the options and let information like price and description be connected directly to each choice.

Tiered Seating with a Set Layout

Each showing creates 36 seats: 6 VIP (Row A) 12 Premium (Rows B-C) 12 Executive (Rows D-E) and 6 Normal (Row F). This looks like a cinema and helps show the seats in a visual map.

UUID-Based Ticket IDs

Ticket IDs use short versions of UUIDs (like TKT-0C4627) to make sure they are unique all over the world without needing a database to keep track of numbers.

Two Types of Promo Codes

Codes that never change (WELCOME10, FESTIVAL20) stay the same. Can only be used once per session. Codes that are made in bulk (BULK20-XXXX) are created when someone books 5 or more seats. This shows how to handle both kinds of discounts.

Stream API for Working, with Data

To find movies, showings and seats the Java Stream API is used of regular loops. This makes working with data clear and fits the way modern Java is written.

Switch Expressions (Java 17 and newer)

The parts that handle menus use Java 17s switch (case "1" ->...) to make the code shorter and avoid mistakes.

9. Implementation Details

9.1 Technology Stack

Aspect	Detail
Language	Java 17 (JDK 17+)
Build	Direct javac compilation
IDE	IntelliJ IDEA
Dependencies	None – Java Standard Library only
Key APIs	Scanner, UUID, Stream, Collections
Design Patterns	Controller Pattern, Enum Strategy

9.2 File Structure

MovieBookingSystem/

```
|— src/
    |— MultiplexBookingSystem.java
|— out/production/      (compiled .class files)
|— MovieBookingSystem.iml (IntelliJ project)
|— .gitignore
```

9.3 Billing Logic

1. Seat Subtotal = Sum of selected seats' tier prices
2. Snack Subtotal = Sum of selected snack prices
3. Sub Total = Seat Subtotal + Snack Subtotal
4. Discount = Sub Total x Promo Code Rate (if valid)
5. After Discount = Sub Total - Discount
6. GST = After Discount x 18%
7. Convenience Fee = Rs. 35.00 (flat)
8. Grand Total = After Discount + GST + Fee

9.4 Seed Data

The app pre-loads 8 movies and 25 shows across 8 screens:

- Trending (5-6 shows each): Dune: Part Two, Oppenheimer, Avatar
- Popular (3 shows each): John Wick 4, Spider-Man: Spider-Verse
- Classics (1-2 shows each): The Dark Knight, Inception, Interstellar

10. Screenshots / Results

```

C:\Windows\System32\cmd.exe
----- USER DASHBOARD -----
1. Book Tickets (Browse Movies & Schedules)
2. View My Tickets (Wallet)
3. Cancel a Ticket
0. Back to Main Menu / Logout
Select Action: 1

```

User Dashboard

All 8 currently playing movies are displayed with IDs and genres. After selecting Dune: Part Two (M04), the 6 available showtimes are listed.

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem
--- Now Showing in Theaters ---
[M01] The Dark Knight (Action/Classic)
[M02] Inception (Sci-Fi/Classic)
[M03] Interstellar (Sci-Fi/Classic)
[M04] Dune: Part Two (Sci-Fi/Trending)
[M05] Avatar: The Way of Water (Fantasy/Trending)
[M06] Spider-Man: Across the Spider-Verse (Animation/Popular)
[M07] Oppenheimer (Biography/Trending)
[M08] John Wick: Chapter 4 (Action/Popular)

Enter Movie ID to view schedules (or 0 to back): M04

--- Schedules for: Dune: Part Two ---
[SH101] Screen 1 | 09:00 AM
[SH102] Screen 2 | 11:30 AM
[SH103] Screen 1 | 02:30 PM
[SH104] Screen 3 | 05:00 PM
[SH105] Screen 1 | 08:00 PM
[SH106] Screen 2 | 10:30 PM

Enter Show ID to book (or 0 to back): SH101

```

Movie Catalogue & Showtimes

The visual seat map displays all 36 seats organized by tier with real-time availability. The user selects seats A1 and A2 (VIP class at Rs. 500 each).

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

===== [ SCREEN 1 ] =====

--- VIP CLASS (Rs. 500.0) ---
[ A1] [ A2] [ A3] [ A4] [ A5] [ A6]

--- PREMIUM CLASS (Rs. 350.0) ---
[ B1] [ B2] [ B3] [ B4] [ B5] [ B6]
[ C1] [ C2] [ C3] [ C4] [ C5] [ C6]

--- EXECUTIVE CLASS (Rs. 250.0) ---
[ D1] [ D2] [ D3] [ D4] [ D5] [ D6]
[ E1] [ E2] [ E3] [ E4] [ E5] [ E6]

--- NORMAL CLASS (Rs. 150.0) ---
[ F1] [ F2] [ F3] [ F4] [ F5] [ F6]
=====

Enter seat numbers separated by space (e.g., B1 B2 C4): A1 A2

```

Seat Map View

The snack bar menu lets users add food items. After ordering, the user applies the WELCOME10 promo code for a 10% discount.

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

--- SNACK BAR ---
1. Large Caramel Popcorn      - Rs. 220.00
2. Jalapeno Cheese Nachos    - Rs. 180.00
3. Fountain Coke              - Rs. 120.00
4. Mineral Water              - Rs. 60.00
0. Proceed to Checkout
Select item to add: 1
Added: POPCORN

--- SNACK BAR ---
1. Large Caramel Popcorn      - Rs. 220.00
2. Jalapeno Cheese Nachos    - Rs. 180.00
3. Fountain Coke              - Rs. 120.00
4. Mineral Water              - Rs. 60.00
0. Proceed to Checkout
Select item to add: 0

Enter Promo Code (Press Enter to skip): WELCOME10
Promo applied! You saved Rs. 122.00

Processing Payment and Generating Final Invoice...
PAYMENT SUCCESSFUL!

```

Snack Bar & Promo Code

A detailed ticket receipt showing all billing components: seat subtotal, snack subtotal, discount, GST (18%), convenience fee (Rs. 35), and grand total.

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

+-----+
|              CONFIRMED TICKET              |
+-----+
| Ticket ID : TKT-0C4627                      |
| Movie    : Dune: Part Two                  |
| Screen   : Screen 1      | Time: 09:00 AM  |
| Seats    : A1, A2                          |
| Snacks    : POPCORN                        |
+-----+
| Seats Subtotal      : Rs.    1000.00      |
| Snacks Subtotal     : Rs.     220.00      |
| Discount Applied    :-Rs.     122.00      |
+-----+
| GST (18%)           : Rs.     197.64      |
| Convenience Fee     : Rs.      35.00      |
+-----+
| GRAND TOTAL         : Rs.    1330.64      |
+-----+

```

Confirmed Ticket / Invoice

The admin portal provides access to analytics, movie management, and show scheduling. Option 1 loads the analytics view.

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

=====
MULTIPLEX MOVIE BOOKING SYSTEM
=====
1. Login as User
2. Login as Admin
0. Exit Application
Select Portal: 2

----- ADMIN DASHBOARD -----
1. View Multiplex Analytics
2. Add New Movie
3. Add New Show
4. Remove Movie
0. Back to Main Menu / Logout
Select Action: 1

```

Admin Portal & Dashboard

Per-show performance data grouped by movie showing seats sold, revenue per show, movie subtotals, and a grand platform revenue total.

```

C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

===== MULTIPLEX ANALYTICS =====

MOVIE: DUNE: PART TWO
[SH101] Screen 1 | 09:00 AM | Seats Sold: 02/36 | Rev: Rs. 1330.64
[SH102] Screen 2 | 11:30 AM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH103] Screen 1 | 02:30 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH104] Screen 3 | 05:00 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH105] Screen 1 | 08:00 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH106] Screen 2 | 10:30 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
=> Subtotal for Dune: Part Two: Rs. 1330.64

MOVIE: THE DARK KNIGHT
[SH122] Screen 8 | 10:30 AM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH123] Screen 8 | 06:00 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
=> Subtotal for The Dark Knight: Rs. 0.00

MOVIE: OPPENHEIMER
[SH107] Screen 5 | 09:30 AM | Seats Sold: 00/36 | Rev: Rs. 0.00
[SH108] Screen 5 | 01:00 PM | Seats Sold: 00/36 | Rev: Rs. 0.00
=> Subtotal for Oppenheimer: Rs. 0.00

=====
GRAND PLATFORM REVENUE: Rs. 1330.64
=====

```

Revenue Analytics

The cancellation flow displays active tickets, accepts a ticket ID, frees the booked seats, and processes a full refund.

```
C:\Windows\System32\cmd.exe - java MultiplexBookingSystem

----- USER DASHBOARD -----
1. Book Tickets (Browse Movies & Schedules)
2. View My Tickets (Wallet)
3. Cancel a Ticket
0. Back to Main Menu / Logout
Select Action: 3

--- Your Active Tickets ---

+-----+
| CONFIRMED TICKET |
+-----+
| Ticket ID : TKT-0C4627 |
| Movie      : Dune: Part Two |
| Screen     : Screen 1      | Time: 09:00 AM |
| Seats      : A1, A2        |
+-----+

Enter the exact Ticket ID to cancel (or 0 to exit): TKT-0C4627

Ticket TKT-0C4627 cancelled successfully. Rs. 1330.64 refunded.
```

Ticket Cancellation & Refund

11. Testing Approach

Testing was done using black-box testing and scripted input testing. Every feature was checked on its own. Then tested as part of the full end-to-end process.

11.1 Test Cases

Test ID	Test Case	Input / Action	Expected Result	Status
TC-01	Main Menu Navigation	Select each portal option	Correct portal opens or app exits	Pass
TC-02	Browse Movies	Login as User, select Book Tickets	All 8 movies displayed	Pass
TC-03	View Showtimes	Enter valid movie ID (M04)	6 shows listed for Dune	Pass
TC-04	Invalid Movie ID	Enter M99	Error: Invalid Movie ID	Pass
TC-05	Seat Map Display	Select valid show ID	36 seats in tiered layout	Pass
TC-06	Book Available Seats	Select A1 A2	Seats booked, ticket generated	Pass
TC-07	Book Already Booked Seat	Try re-booking A1	Error: seat already booked	Pass
TC-08	Valid Promo Code	Enter WELCOME10	10% discount applied	Pass
TC-09	Reused Promo Code	Reuse WELCOME10	Error: already claimed	Pass
TC-10	Invalid Promo Code	Enter FAKEPROMO	No discount, message shown	Pass
TC-11	Snack Bar Ordering	Add Popcorn and Coke	Items added, totals correct	Pass
TC-12	Billing Calculation	2 VIP + WELCOME10	GST + fee calculated correctly	Pass
TC-13	Bulk Reward Code	Book 5+ seats	BULK20-XXXX code generated	Pass
TC-14	View Wallet	After booking, view wallet	Active tickets displayed	Pass
TC-15	Cancel Ticket	Enter valid ticket ID	Seats freed, amount refunded	Pass
TC-16	Admin - Add Movie	Add new movie	Movie added with auto ID	Pass
TC-17	Admin - Add Show	Create show for movie	Show created with new ID	Pass
TC-18	Admin - Remove Movie	Remove movie with shows	Cascade delete executed	Pass
TC-19	Admin - Analytics	View after bookings	Revenue data displayed	Pass
TC-20	Empty Wallet	View wallet, no bookings	Message: wallet is empty	Pass

12. Challenges Faced

Console-Based Seat Map Rendering

Creating a seat map that was easy to read and properly aligned using plain text was one of the hardest tasks. Each row had to have the spacing and the change between different tiers needed to be easy to see. Making sure the printf formatting worked for both booked [XX] seats and available seats meant doing calculations for field widths.

Promo Code Lifecycle Management

Managing the life of promo codes was complicated. Permanent codes had to be marked as used but not removed. Dynamic bulk codes had to be created saved, checked, applied and taken back if someone canceled. Keeping track of three maps at the same time needed good state control.

Cascading Deletions

When an admin deletes a movie the system goes through all the shows and tickets. Using removelf() with the conditions while making sure there was no conflict was important.

Input Validation Without Crashes

Every place where someone interacts with the system needed validation. Wrong menu choices, IDs, seats that were already booked and incorrect promo codes all had to be handled in a way that didn't crash the system. Each error needed a message to help the user understand what went wrong.

Revenue Tracking Accuracy

Keeping the revenue numbers correct, during bookings and cancellations was very important. The total amount including GST and fees had to be exactly right when a booking was cancelled.

13. Learnings & Key Takeaways

Object-Oriented Design Matters

Breaking the problem into entity classes made the code much easier to manage. Each class holds its data and behavior so changes stay isolated and predictable.

Enums Are Powerful in Java

Using enums that carry data removed the need for configuration maps. Enums give type safety, simple iteration and self-documenting code.

Stream API Improves Readability

Replacing loops with Stream operations made data processing concise. Finding a movie by ID or calculating revenue turned into one-liners.

User Experience Matters in CLI

Investing in output—bordered tickets, tiered seat maps clear errors—dramatically improved the user experience. A well-designed CLI can be as engaging as a GUI.

State Consistency Is Non-Negotiable

Every booking must update seats, revenue and wallet. Every cancellation must reverse all changes. Revoke promo codes. Missing any step would cause bugs in analytics or future bookings.

Seed Data Creates a Better Demo

Pre-loading eight movies and twenty-five shows made the app feel real, from the run enabling meaningful demonstrations without manual setup.

14. Future Enhancements

Database Integration

Replace in-memory storage with MySQL/PostgreSQL using JDBC or JPA/Hibernate for data persistence.

User Authentication

Implement login with username/password, registration, and per-user ticket wallets.

GUI / Web Interface

Build a JavaFX desktop app or Spring Boot web app with interactive seat selection.

Payment Gateway

Integrate Razorpay or Stripe for actual payment processing.

Email / SMS Notifications

Send booking confirmations via email (JavaMail) or SMS (Twilio).

Seat Locking / Timeout

Temporarily reserve seats during booking to prevent double-booking.

Dynamic Pricing

Increase prices as occupancy rises, similar to airline pricing.

Review & Rating System

Allow users to rate and review movies after watching.

15. References

- [1] Oracle Java Documentation – Java SE 17 API Reference.
<https://docs.oracle.com/en/java/javase/17/docs/api/>
- [2] Bloch, J. (2018). Effective Java, 3rd Edition. Addison-Wesley Professional.
- [3] Oracle Java Tutorials – Collections Framework.
<https://docs.oracle.com/javase/tutorial/collections/>
- [4] Oracle Java Tutorials – Lambda Expressions.
<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>
- [5] Oracle Java Tutorials – Aggregate Operations (Streams).
<https://docs.oracle.com/javase/tutorial/collections/streams/>
- [6] Baeldung – Guide to Java Enums. <https://www.baeldung.com/a-guide-to-java-enums>
- [7] GeeksforGeeks – Scanner Class in Java. <https://www.geeksforgeeks.org/scanner-class-in-java/>
- [8] BookMyShow – Inspiration for seat layout and booking flow.
<https://www.bookmyshow.com/>
- [9] IntelliJ IDEA Documentation. <https://www.jetbrains.com/idea/documentation/>
- [10] UML Diagrams – Notation reference. <https://www.uml-diagrams.org/>